Analyzing Algorithms

- Comparison between various candidate algorithms
- Why not implement and test?
  - Caveats:
    - too many algorithms
      - depends on input size, how inputs are chosen
- We will count the number of basic operations like addition, comparison, etc.
- And see how this number grows as a function of the input size. This measure is independent of the choice of the machine.

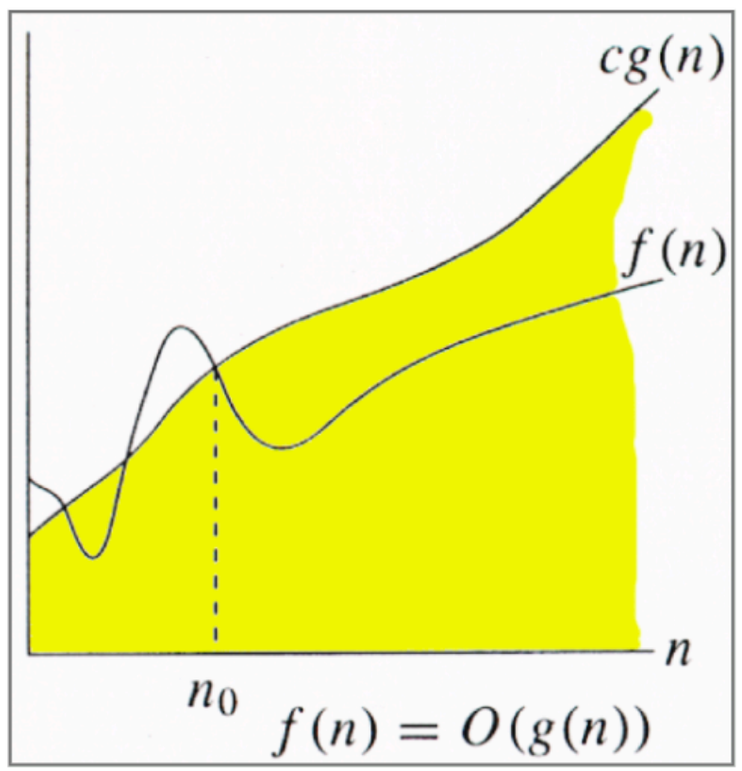
O(·) notation

- For input size n, running time $f(n)$.

- We say $f(n) = O(g(n))$ if there exist constants $c > 0$, $n_0 > 0$ such that
  $0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0$. (Sometimes we will use $T(n)$ instead of $g(n)$.)

- O-notation is an *upper-bound* notation.

- It makes **no sense** to say $f(n)$ is <u>at least</u> $O(n)$.



O(·) notation

- Why do we ignore constant factors?

- Because it's (sometimes) not possible to find the precise constant factor. Various basic operations do not take the same amount of time.

- Is *O(n)* always better than *O(n log n)?*

  - For large enough inputs, yes. But, depending on the hidden constant factors, it's possible that *O(n log n)* algorithm is faster on reasonable size inputs.

Worst case analysis

- Worst case bound: running time guarantee for all possible inputs of a size.

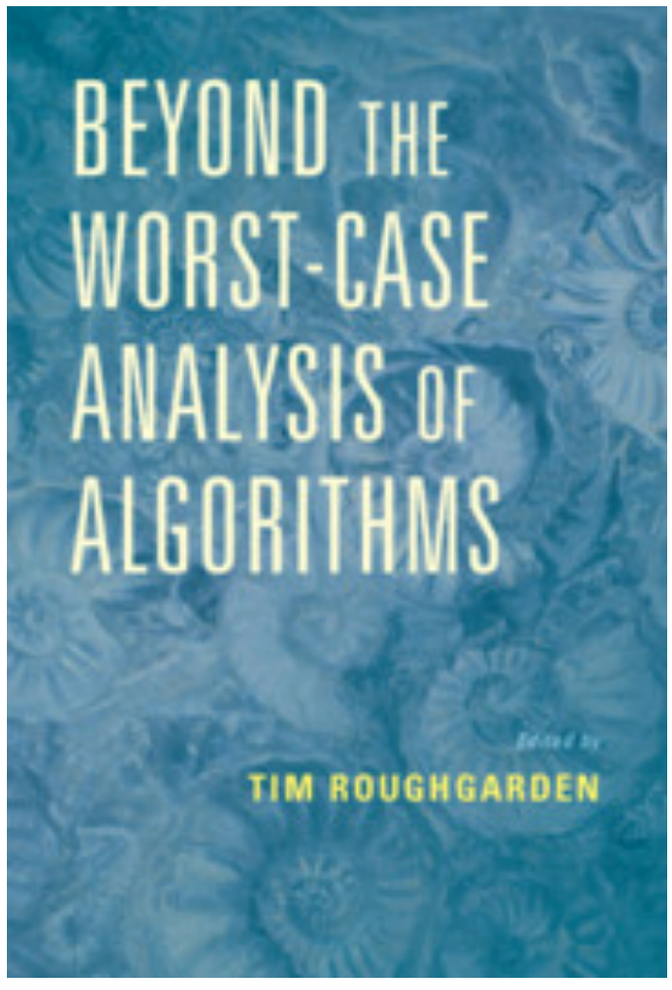
- There could be algorithms which are slow on a few pathological instances, but otherwise quite fast.

- Why not analyze only for “real world inputs”?

  - It’s not clear how to model “real world inputs”.

- For many algorithms, we are able to give worst case bounds.

Worst case analysis



Out of scope of this course

Describing algorithms

- Find the maximum sum subarray of length k

```
s = 0;
for (i=0, i < k, i++)  s=s+A[i];
m = s;
for (i=0, i < n-k, i++){
      s = s - A[i] + A[k+i];
      if (s > m) m = s;
}
return m;
```

Precise, but hard to understand. Error prone.

Compute the sum of first k numbers. We will go over all length k subarrays from left to right. In an iteration, update the sum by subtracting the first number of the current array and adding the number following the last one. If the sum is larger than the maximum seen so far, we update the maximum.

Not precise. Open to multiple interpretations.

```
s ← sum of first k numbers;
for (i ← 0 to n-k-1){
      s ← s - A[i] + A[k+i];
      m ← max(m, s);
}
return m;
```

Somewhere in the middle.

Describing algorithms

- Two sorted (increasing) arrays A and B of length n. Count pairs (a,b) such that $a \in A$ and $b \in B$ and $a < b$

```
j=0; count = 0;
for (i=0, i < n, i++){
      while (A[i] >= B[j]) j++;
      count=count + n - j;
}
return count;
```

```
j ← 0; count ← 0;
for (i ← 0 to n-1){
      keep increasing j till we get B[j] > A[i];
      count ← count + n - j;
}
return count;
```

O(·) notation

- True or False?
- $2n+3$ is $O(n^2)$

- $1^2 + 2^2 + \ldots + (n-1)^2 + n^2$ is $O(n^2)$

- $1 + 1/2 + 1/3 + \ldots + 1/n$ is $O(\log n)$

- n^n is $O(2^n)$

O(·) notation

- True or False?

- $2n+3$ is $O(n^2)$
  - True
- $1^2 + 2^2 + \ldots + (n-1)^2 + n^2$ is $O(n^2)$
  - False (it is $\Theta(n^3)$)
- $1 + 1/2 + 1/3 + \ldots + 1/n$ is $O(\log n)$
  - True
- n^n is $O(2^n)$
  - False

O(·) notation

- True or False?
- 2^{3n} is $O(2^n)$

- $(n+1)^3$ is $O(n^3)$

- $(n+\sqrt{n})^2$ is $O(n^2)$

- $log(n^3)$ is $O(log\ n)$

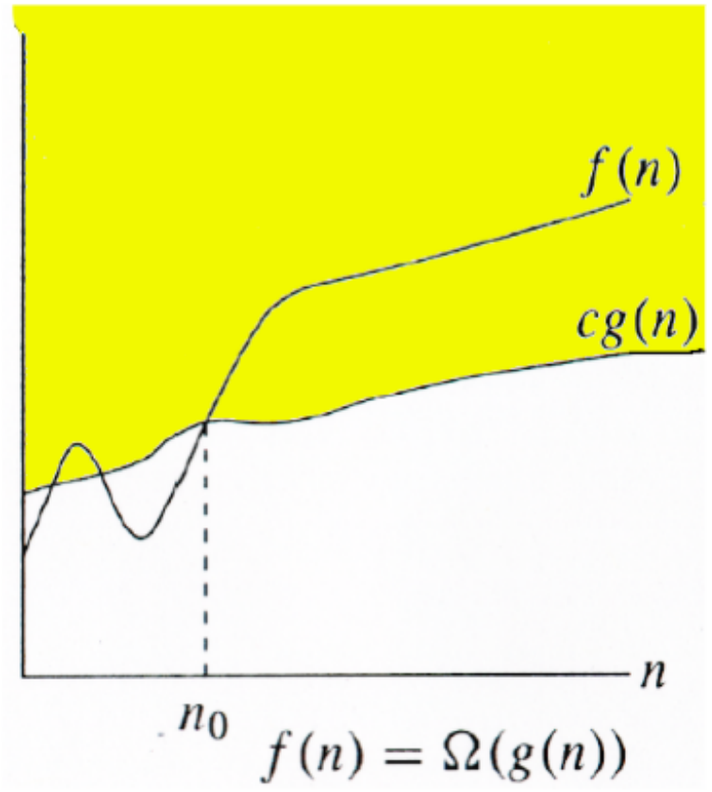
O(·) notation

- True or False?
- 2^{3n} is $O(2^n)$
  - False
- $(n+1)^3$ is $O(n^3)$
  - True
- $(n+\sqrt{n})^2$ is $O(n^2)$
  - True
- $log(n^3)$ is $O(log\ n)$
  - True

Ω-notation (lower bound)

We write $f(n) = \Omega(g(n))$ if there exist constants $c > 0$, $n_0 > 0$ such that

$0 \leq c.g(n) \leq f(n)$ for all $n \geq n_0$.

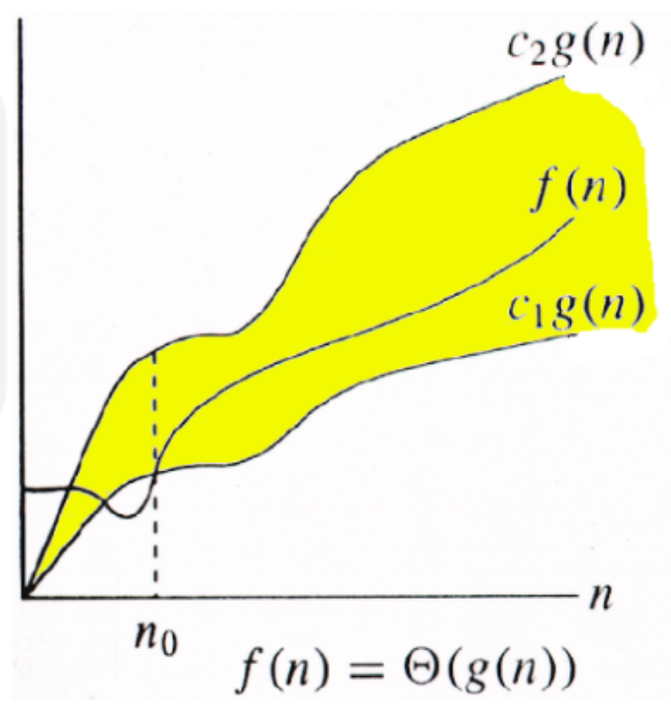


- Example:
  - $n^2 - n = \Omega(n^2)$
  - $0 \leq \frac{1}{2}n^2 \leq (n^2 - n)$, for $n \geq 2$.
  - Hence, $n^2 - n = \Omega(n^2)$.

Θ-Notation: Tight Bounds

$\Theta(g(n)) = \{ f(n)$: there exist positive constants c_1, c_2 and n_0 such that, $0 \le c_1 . g(n) \le f(n) \le c_2 . g(n)$ for all $n \ge n_0 \}$

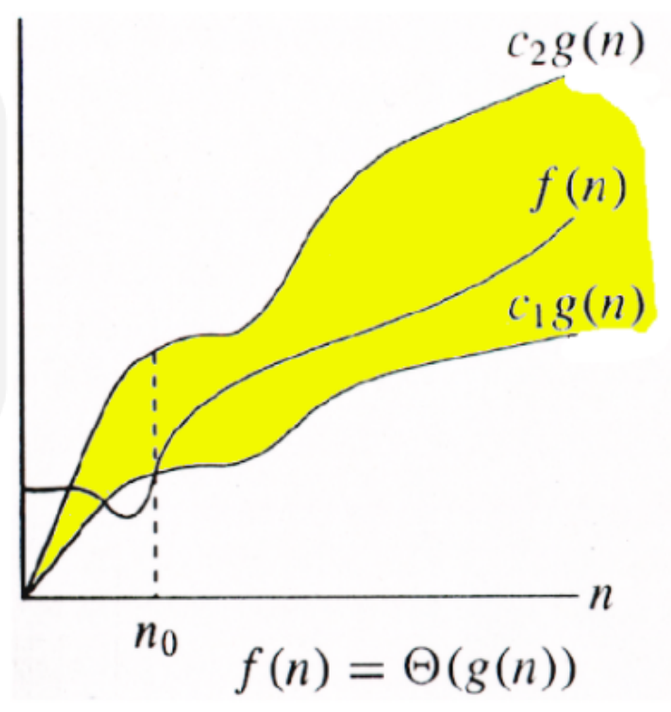
- Example:
- $n^2 - n = \Omega(n^2)$
- $0 \le \frac{1}{2} n^2 \le (n^2 - n) \le n^2$, for $(c_1 = \frac{1}{2}, c_2 = 1, n_0 = 2)$
- Hence, $n^2 - n = \Theta(n^2)$.



Θ-Notation: Tight Bounds

$\Theta(g(n)) = \{ f(n)$: there exist positive constants c_1, c_2 and n_0 such that, $0 \le c_1 . g(n) \le f(n) \le c_2 . g(n)$ for all $n \ge n_0 \}$

- Example:
  - $n^2 - n = \Omega(n^2)$
  - $0 \le \frac{1}{2} n^2 \le (n^2 - n) \le n^2$, for $(c_1 = \frac{1}{2}, c_2 = 1, n_0 = 2)$
  - Hence, $n^2 - n = \Theta(n^2)$.



Prove, $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$

o-notation and ω-notation

O-notation and Ω-notation are like $\leq$ and $\geq$.

o-notation and ω-notation are like $<$ and $>$.

$o(g(n)) = \{ f(n) :$ for any constant $c > 0$,
there is a constant $n_0 > 0$ such that $0 \leq f(n) < cg(n)$ for all $n \geq n_0$ $\}$

$$n = o(n^2), \text{ for } c_2 = 2, n_0 = 1$$

$\omega(g(n)) = \{ f(n) :$ for any constant $c > 0$,
there is a constant $n_0 > 0$ such that $0 \leq cg(n) < f(n)$ for all $n \geq n_0$ $\}$

$$n^2 - n = \omega(n),$$

Principles of algorithm design

First principle: reducing to a subproblem

- Subproblem: same problem on a smaller input
  - Assume that you have already built the solution for the subproblem and using that try to build the solution for the original problem.
- Subproblem will be solved using the same strategy.
- Implementation: recursive or iterative

First principle: reducing to a subproblem

- Example 1: finding minimum value in an array.
- Suppose we have already found minimum among first $n-1$ numbers, say min_{n-1}
- $min_n = \text{minimum}(min_{n-1}, A[n])$
- Iterative implementation:
  Go over the array from 1 to n and maintain a variable min
- Invariant: after seeing i numbers, min will be the minimum among first i numbers.
- $min_i = \text{minimum}(min_{i-1}, A[i])$

First principle: reducing to a subproblem

- min_i = minimum(min_{i-1}, $A[i]$)

- Iterative implementation

```
min ← A[1];
for (i ← 2 to n)
    min ← minimum(min, A[i])
```

- Recursive implementation

```
f(A, i):
    if i=1 return A[1];
    else return minimum( f(A, i-1), A[i]);
Compute f(A, n);
```

Maximum subarray sum

- Given an integer array with positive/negative numbers. Find the subarray with maximum possible sum.

- 1, 2, -5, 4, -6, 8, 7, -3, 2, 10, 3, -7, 4, 2

- $O(n^2)$ algorithm

- For every choice of starting point,
  go over all choices of end points and maintain the sum between starting and end points.

- Maintain the *max_sum* value by comparing with the current sum

Maximum subarray sum

O(n^2) algorithm:

```
max_sum ← 0;
for (start ← 1 to n){
        curr_sum ← 0;
        for (end ← start to n){
            curr_sum ← curr_sum + A[end];  // update the current sum
            max_sum ← maximum(curr_sum, max_sum);
        }
}
```

Max subarray sum: subproblem

- Suppose we have already found the maximum subarray sum for $A[1\ldots n\text{-}1]$, say max_sum_{n-1}
- How do we compute max_sum_n
- Two kinds of subarrays of A:
  1. subarrays of $A[1\ldots n\text{-}1]$
  2. subarrays of A which end at n
- max_sum_n = Maximum(max_sum_{n-1} ,
  $Sum(1 \ldots n)$,
  $Sum(2 \ldots n)$,
  $\vdots$
  $Sum(n \ldots n)$,) } O(n)

$$T(n) = T(n\text{-}1) + O(n) \qquad \Rightarrow T(n) = O(n^2)$$

Max subarray sum: subproblem

- Improvement ?

- Observation:

$$Sum(1 \ldots n) = Sum(1 \ldots n\text{-}1) + A[n],$$
$$Sum(2 \ldots n) = Sum(2 \ldots n\text{-}1) + A[n],$$
$$\vdots$$
$$Sum(n\text{-}1 \ldots n) = Sum(n\text{-}1 \ldots n\text{-}1) + A[n],$$

- Maximum($Sum(1 \ldots n), Sum(2 \ldots n), \ldots, Sum(n\text{-}1 \ldots n)$)

$$=$$

Maximum($Sum(1 \ldots n\text{-}1), Sum(2 \ldots n\text{-}1), \ldots, Sum(n\text{-}1 \ldots n\text{-}1)$) + $A[n]$

- We have converted it to another problem on first $n\text{-}1$ numbers

Max subarray sum: subproblem

- Improvement ?

- Observation:

$$Sum(1 \ldots n) = Sum(1 \ldots n\text{-}1) + A[n],$$
$$Sum(2 \ldots n) = Sum(2 \ldots n\text{-}1) + A[n],$$
$$\vdots$$
$$Sum(n\text{-}1 \ldots n) = Sum(n\text{-}1 \ldots n\text{-}1) + A[n],$$

- Maximum($Sum(1 \ldots n), Sum(2 \ldots n), \ldots, Sum(n\text{-}1 \ldots n)$)

$$=$$

Maximum($Sum(1 \ldots n\text{-}1), Sum(2 \ldots n\text{-}1), \ldots, Sum(n\text{-}1 \ldots n\text{-}1)$) + $A[n]$

- We have converted it to another problem on first $n\text{-}1$ numbers

Max subarray sum: subproblem

- Improvement ?
- Observation:

$$Sum(1 \dots n) = Sum(1 \dots n\text{-}1) + A[n],$$
$$Sum(2 \dots n) = Sum(2 \dots n\text{-}1) + A[n],$$
$$\vdots$$
$$Sum(n\text{-}1 \dots n) = Sum(n\text{-}1 \dots n\text{-}1) + A[n],$$

Push it to the subproblem

- Maximum($Sum(1 \dots n), Sum(2 \dots n), \dots, Sum(n\text{-}1 \dots n)$)

=

Maximum($Sum(1 \dots n\text{-}1), Sum(2 \dots n\text{-}1), \dots, Sum(n\text{-}1 \dots n\text{-}1)$) + $A[n]$

- We have converted it to another problem on first *n-1* numbers

Asking subproblem to do more

- Subproblem: max_sum_{n-1} and $max\_suffix\_sum_{n-1}$

- $max\_suffix\_sum_{n-1}$ is defined as
  Maximum(*Sum(1 … n-1), Sum(2 … n-1), … Sum(n-1 … n-1))*

- max_sum_n = Maximum(max_sum_{n-1} ,
  Sum(1 … n-1) + A[n],
  Sum(2 … n-1) + A[n],
  ⋮
  Sum(n-1 … n-1) + A[n],
  A[n])

- = Maximum(max_sum_{n-1} ,
  $max\_suffix\_sum_{n-1}$ *+ A[n],*
  A[n])

Asking subproblem to do more

- Subproblem: max_sum_{n-1} and $max\_suffix\_sum_{n-1}$

- $max\_sum_n = \text{Maximum}(max\_sum_{n-1}, max\_suffix\_sum_{n-1} + A[n], A[n])$

- We are asking the subproblem to compute *max_suffix_sum* for size *n-1*
  So, we also need to compute *max_suffix_sum* for size *n*

- $max\_suffix\_sum_n = ?$

- $max\_suffix\_sum_n = \text{Maximum}(max\_suffix\_sum_{n-1} + A[n], A[n])$

$T(n) = T(n\text{-}1) + O(1) \quad \Rightarrow T(n) = O(n)$

Maximum subarray sum

O(n) algorithm:

$max\_sum \leftarrow 0$; $max\_suffix\_sum \leftarrow 0$;

for ($i \leftarrow 1$ to n){

$max\_sum \leftarrow$ maximum(max_sum,
$max\_suffix\_sum + A[i]$,
$A[i]$);

$max\_suffix\_sum \leftarrow$ maximum($max\_suffix\_sum + A[i]$, $A[i]$);

}

Alternate implementation

```
max_sum ← 0;   max_suffix_sum ← 0;

for (i ← 1 to n){

        max_suffix_sum ← maximum(A[i], max_suffix_sum +A[i]);

        max_sum ← maximum(max_suffix_sum, max_sum);

        }
```

- Here we are updating the two variables in a different order.

Reducing to a subproblem

- When solving a problem recursively/inductively, it is sometimes useful to solve a more general problem

- Stronger induction hypothesis

Exercises

- Given share prices for n days
  $p_1, p_2, \ldots, p_n$
- You have to buy it on one of the days and sell it on a later day.
- Maximum profit possible in $O(n)$?
- $max_{j>i}(p_j - p_i)$

Exercises

- There is a party with *n* people, among them there is 1 celebrity.
- A celebrity is someone who is known to everyone, but she does not know anyone.
- you ask the any person *i* if they know person *j*.
- Can you the find the celebrity in *O(n)* queries?